WIN SEM 2025-2026

LAB EXPERIMENT REPORT

Student Registration Number: 23BCE7691

Student Name: Gaurav Sudish

Course Name: Web Technologies

Lab Slot: L37-L38

Experiment No: 6

Use MongoDB to implement the following DB operations

1. Create a database called ‘vehicles’ and write a MongoDB query to select database as “vehicles”.

```
> use vehicles
< switched to db vehicles
```

2. Write a MongoDB query to display all the databases.

```
> show dbs
< animal        112.00 KiB
  saathi        308.00 KiB
  sample_mflix  116.39 MiB
  test          408.00 KiB
  vehicles      144.00 KiB
```

3. Create a collection called ‘two_wheelers’. (use capping) and Create a collection called ‘four_wheelers’.

```
> db.createCollection("two_wheelers", {capped: true, size: 1048576})
< { ok: 1 }
> db.createCollection("four_wheelers")
< { ok: 1 }
```

4. Add 5 two-wheeler details to the collection named ‘two_wheelers’. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
> db.two_wheelers.insertMany([
    {
      bike_name: "Pulsar NS200", model: "gear", category: "200cc",
      colors_available: ["red", "black", "blue"], manufacturer: "Bajaj",
      performance: 8.7, timestamp: "2024-01-15", price: 145000
    },
    {
      bike_name: "Shine", model: "gear", category: "125cc",
      colors_available: ["blue", "black"], manufacturer: "Honda",
      performance: 7.5, timestamp: "2024-02-10", price: 85000
    },
    {
      bike_name: "Activa 6G", model: "gearless", category: "110cc",
      colors_available: ["red", "white", "black"], manufacturer: "Honda",
      performance: 7.2, timestamp: "2024-03-05", price: 78000
    },
    {
      bike_name: "Unicorn", model: "gear", category: "150cc",
      colors_available: ["red", "matte black"], manufacturer: "Honda",
      performance: 8.2, timestamp: "2023-12-20", price: 112000
    },
    {
      bike_name: "Hornet 2.0", model: "gear", category: "200cc",
      colors_available: ["neon green", "red"], manufacturer: "Honda",
      performance: 8.9, timestamp: "2024-04-01", price: 165000
    }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b137c55befb9e591937c4c'),
      '1': ObjectId('69b137c55befb9e591937c4d'),
      '2': ObjectId('69b137c55befb9e591937c4e'),
      '3': ObjectId('69b137c55befb9e591937c4f'),
      '4': ObjectId('69b137c55befb9e591937c50')
    }
  }
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
> db.four_wheelers.insertMany([
    {
      vehicle_name: "Swift", model: "own", category: "car",
      variants: ["vxi", "zxi", "petrol"], manufacturer: "Maruti",
      performance: 7.8, timestamp: "2024-03-20", price: 750000
    },
    {
      vehicle_name: "Bolero", model: "commercial", category: "lorry",
      variants: ["diesel", "SLE"], manufacturer: "Mahindra",
      performance: 6.5, timestamp: "2024-01-10", price: 950000
    },
    {
      vehicle_name: "Innova Crysta", model: "own", category: "car",
      variants: ["petrol", "diesel", "GX"], manufacturer: "Toyota",
      performance: 8.1, timestamp: "2024-02-15", price: 2200000
    },
    {
      vehicle_name: "Pro 2000", model: "commercial", category: "mini truck",
      variants: ["diesel"], manufacturer: "Eicher",
      performance: 7.0, timestamp: "2023-11-30", price: 1250000
    },
    {
      vehicle_name: "Boss 1115", model: "commercial", category: "heavy truck",
      variants: ["diesel", "CNG"], manufacturer: "Ashok Leyland",
      performance: 6.8, timestamp: "2024-04-10", price: 2800000
    }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b137eb5befb9e591937c51'),
      '1': ObjectId('69b137eb5befb9e591937c52'),
      '2': ObjectId('69b137eb5befb9e591937c53'),
      '3': ObjectId('69b137eb5befb9e591937c54'),
      '4': ObjectId('69b137eb5befb9e591937c55')
    }
  }
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
db.four_wheelers.find().pretty()
< {
    _id: ObjectId('69b137eb5befb9e591937c51'),
    vehicle_name: 'Swift',
    model: 'own',
    category: 'car',
    variants: [
      'vxi',
      'zxi',
      'petrol'
    ],
    manufacturer: 'Maruti',
    performance: 7.8,
    timestamp: '2024-03-20',
    price: 750000
  }
  {
    _id: ObjectId('69b137eb5befb9e591937c52'),
    vehicle_name: 'Bolero',
    model: 'commercial',
    category: 'lorry',
    variants: [
      'diesel',
      'SLE'
    ],
    manufacturer: 'Mahindra',
    performance: 6.5,
    timestamp: '2024-01-10',
    price: 950000
  }
```

```
{
  _id: ObjectId('69b137eb5befb9e591937c53'),
  vehicle_name: 'Innova Crysta',
  model: 'own',
  category: 'car',
  variants: [
    'petrol',
    'diesel',
    'GX'
  ],
  manufacturer: 'Toyota',
  performance: 8.1,
  timestamp: '2024-02-15',
  price: 2200000
}
{
  _id: ObjectId('69b137eb5befb9e591937c54'),
  vehicle_name: 'Pro 2000',
  model: 'commercial',
  category: 'mini truck',
  variants: [
    'diesel'
  ],
  manufacturer: 'Eicher',
  performance: 7,
  timestamp: '2023-11-30',
  price: 1250000
}
```

```
{
  _id: ObjectId('69b137eb5befb9e591937c55'),
  vehicle_name: 'Boss 1115',
  model: 'commercial',
  category: 'heavy truck',
  variants: [
    'diesel',
    'CNG'
  ],
  manufacturer: 'Ashok Leyland',
  performance: 6.8,
  timestamp: '2024-04-10',
  price: 2800000
}
> db.two_wheelers.find().pretty()

< {
  _id: ObjectId('69b137c55befb9e591937c4c'),
  bike_name: 'Pulsar NS200',
  model: 'gear',
  category: '200cc',
  colors_available: [
    'red',
    'black',
    'blue'
  ],
  manufacturer: 'Bajaj',
  performance: 8.7,
  timestamp: '2024-01-15',
  price: 145000
}
```

```
{
  _id: ObjectId('69b137c55befb9e591937c4d'),
  bike_name: 'Shine',
  model: 'gear',
  category: '125cc',
  colors_available: [
    'blue',
    'black'
  ],
  manufacturer: 'Honda',
  performance: 7.5,
  timestamp: '2024-02-10',
  price: 85000
}
{
  _id: ObjectId('69b137c55befb9e591937c4e'),
  bike_name: 'Activa 6G',
  model: 'gearless',
  category: '110cc',
  colors_available: [
    'red',
    'white',
    'black'
  ],
  manufacturer: 'Honda',
  performance: 7.2,
  timestamp: '2024-03-05',
  price: 78000
}
{
  _id: ObjectId('69b137c55befb9e591937c4f'),
  bike_name: 'Unicorn',
  model: 'gear',
  category: '150cc',
  colors_available: [
    'red',
    'matte black'
  ],
  manufacturer: 'Honda',
  performance: 8.2,
  timestamp: '2023-12-20',
  price: 112000
}
```

```
{
  _id: ObjectId('69b137c55befb9e591937c50'),
  bike_name: 'Hornet 2.0',
  model: 'gear',
  category: '200cc',
  colors_available: [
    'neon green',
    'red'
  ],
  manufacturer: 'Honda',
  performance: 8.9,
  timestamp: '2024-04-01',
  price: 165000
}
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
db.four_wheelers.aggregate([
  { $project: { name: "$vehicle_name", price: 1, _id: 0 } }
])
< {
  price: 750000,
  name: 'Swift'
}
{
  price: 950000,
  name: 'Bolero'
}
{
  price: 2200000,
  name: 'Innova Crysta'
}
{
  price: 1250000,
  name: 'Pro 2000'
}
{
  price: 2800000,
  name: 'Boss 1115'
}
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
> db.two_wheelers.find({manufacturer: "Honda"}).pretty()
< {
    _id: ObjectId('69b137c55befb9e591937c4d'),
    bike_name: 'Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [
      'blue',
      'black'
    ],
    manufacturer: 'Honda',
    performance: 7.5,
    timestamp: '2024-02-10',
    price: 85000
  }
  {
    _id: ObjectId('69b137c55befb9e591937c4e'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '110cc',
    colors_available: [
      'red',
      'white',
      'black'
    ],
    manufacturer: 'Honda',
    performance: 7.2,
    timestamp: '2024-03-05',
    price: 78000
  }
  {
    _id: ObjectId('69b137c55befb9e591937c4f'),
    bike_name: 'Unicorn',
    model: 'gear',
    category: '150cc',
    colors_available: [
      'red',
      'matte black'
    ],
    manufacturer: 'Honda',
    performance: 8.2,
    timestamp: '2023-12-20',
    price: 112000
  }
```

```
  {
    _id: ObjectId('69b137c55befb9e591937c50'),
    bike_name: 'Hornet 2.0',
    model: 'gear',
    category: '200cc',
    colors_available: [
      'neon green',
      'red'
    ],
    manufacturer: 'Honda',
    performance: 8.9,
    timestamp: '2024-04-01',
    price: 165000
  }
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
> db.four_wheelers.find({"variants": "diesel"}).pretty()
< {
    _id: ObjectId('69b137eb5befb9e591937c52'),
    vehicle_name: 'Bolero',
    model: 'commercial',
    category: 'lorry',
    variants: [
      'diesel',
      'SLE'
    ],
    manufacturer: 'Mahindra',
    performance: 6.5,
    timestamp: '2024-01-10',
    price: 950000
  }
  {
    _id: ObjectId('69b137eb5befb9e591937c53'),
    vehicle_name: 'Innova Crysta',
    model: 'own',
    category: 'car',
    variants: [
      'petrol',
      'diesel',
      'GX'
    ],
    manufacturer: 'Toyota',
    performance: 8.1,
    timestamp: '2024-02-15',
    price: 2200000
  }
  {
    _id: ObjectId('69b137eb5befb9e591937c54'),
    vehicle_name: 'Pro 2000',
    model: 'commercial',
    category: 'mini truck',
    variants: [
      'diesel'
    ],
    manufacturer: 'Eicher',
    performance: 7,
    timestamp: '2023-11-30',
    price: 1250000
  }
```

```
  {
    _id: ObjectId('69b137eb5befb9e591937c55'),
    vehicle_name: 'Boss 1115',
    model: 'commercial',
    category: 'heavy truck',
    variants: [
      'diesel',
      'CNG'
    ],
    manufacturer: 'Ashok Leyland',
    performance: 6.8,
    timestamp: '2024-04-10',
    price: 2800000
  }
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
> db.two_wheelers.find(
    {performance: {$gt: 5}},
    {bike_name: 1, category: 1, manufacturer: 1, _id: 0}
  ).pretty()
< {
    bike_name: 'Pulsar NS200',
    category: '200cc',
    manufacturer: 'Bajaj'
  }
  {
    bike_name: 'Shine',
    category: '125cc',
    manufacturer: 'Honda'
  }
  {
    bike_name: 'Activa 6G',
    category: '110cc',
    manufacturer: 'Honda'
  }
  {
    bike_name: 'Unicorn',
    category: '150cc',
    manufacturer: 'Honda'
  }
  {
    bike_name: 'Hornet 2.0',
    category: '200cc',
    manufacturer: 'Honda'
  }
```

```
> db.four_wheelers.find(
    {performance: {$gt: 5}},
    {vehicle_name: 1, category: 1, manufacturer: 1, _id: 0}
  ).pretty()
< {
    vehicle_name: 'Swift',
    category: 'car',
    manufacturer: 'Maruti'
  }
  {
    vehicle_name: 'Bolero',
    category: 'lorry',
    manufacturer: 'Mahindra'
  }
  {
    vehicle_name: 'Innova Crysta',
    category: 'car',
    manufacturer: 'Toyota'
  }
  {
    vehicle_name: 'Pro 2000',
    category: 'mini truck',
    manufacturer: 'Eicher'
  }
  {
    vehicle_name: 'Boss 1115',
    category: 'heavy truck',
    manufacturer: 'Ashok Leyland'
  }
```

Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called ‘animal’ and write a MongoDB query to select database as ‘animal’.

```
> use animal
< switched to db animal
```

2. Write a MongoDB query to display all the databases.

```
> show databases
< animal          8.00 KiB
  saathi        308.00 KiB
  sample_mflix  109.88 MiB
  test          384.00 KiB
  vehicles       80.00 KiB
  admin         360.00 KiB
  local          29.68 GiB
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

```
> db.createCollection("wild_animals", {capped: true, size: 1048576})
  db.createCollection("domestic_animals")
< { ok: 1 }
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
> db.wild_animals.insertMany([
    {
      animal_name: "Tiger", nature: "harm",
      favorite_foods: ["deer", "meat"],
      care_taker_name: "Ravi", life_span: 15,
      timestamp: "2024-02-10", expenses: 50000
    },
    {
      animal_name: "Lion", nature: "harm",
      favorite_foods: ["meat", "zebra"],
      care_taker_name: "Kumar", life_span: 14,
      timestamp: "2024-01-20", expenses: 48000
    },
    {
      animal_name: "Elephant", nature: "harmless",
      favorite_foods: ["grass", "leaves"],
      care_taker_name: "Priya", life_span: 60,
      timestamp: "2023-12-15", expenses: 75000
    },
    {
      animal_name: "Bear", nature: "harm",
      favorite_foods: ["honey", "fish"],
      care_taker_name: "Ravi", life_span: 25,
      timestamp: "2024-03-01", expenses: 35000
    },
    {
      animal_name: "Wolf", nature: "harm",
      favorite_foods: ["rabbits", "meat"],
      care_taker_name: "Anil", life_span: 13,
      timestamp: "2024-02-15", expenses: 28000
    }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b13a242c916e330847e3dd'),
      '1': ObjectId('69b13a242c916e330847e3de'),
      '2': ObjectId('69b13a242c916e330847e3df'),
      '3': ObjectId('69b13a242c916e330847e3e0'),
      '4': ObjectId('69b13a242c916e330847e3e1')
    }
  }
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
> db.domestic_animals.insertMany([
    {
      animal_name: "Dog", gender: "male",
      favorite_foods: ["meat", "bones"],
      animal_petname: "Bruno", life_span: 12,
      timestamp: "2024-01-05", expenses: 15000
    },
    {
      animal_name: "Cat", gender: "female",
      favorite_foods: ["fish", "milk"],
      animal_petname: "Whiskers", life_span: 15,
      timestamp: "2024-01-10", expenses: 12000
    },
    {
      animal_name: "Cow", gender: "female",
      favorite_foods: ["grass", "hay"],
      animal_petname: "Lakshmi", life_span: 20,
      timestamp: "2023-11-20", expenses: 25000
    },
    {
      animal_name: "Rabbit", gender: "male",
      favorite_foods: ["carrots", "grass"],
      animal_petname: "Bunny", life_span: 8,
      timestamp: "2024-02-01", expenses: 8000
    },
    {
      animal_name: "Horse", gender: "male",
      favorite_foods: ["grass", "oats"],
      animal_petname: "Thunder", life_span: 25,
      timestamp: "2024-03-10", expenses: 40000
    }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b13a872c916e330847e3e2'),
      '1': ObjectId('69b13a872c916e330847e3e3'),
      '2': ObjectId('69b13a872c916e330847e3e4'),
      '3': ObjectId('69b13a872c916e330847e3e5'),
      '4': ObjectId('69b13a872c916e330847e3e6')
    }
  }
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
> db.wild_animals.find().pretty()
< {
    _id: ObjectId('69b13a242c916e330847e3dd'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [
      'deer',
      'meat'
    ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: '2024-02-10',
    expenses: 50000
  }
  {
    _id: ObjectId('69b13a242c916e330847e3de'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [
      'meat',
      'zebra'
    ],
    care_taker_name: 'Kumar',
    life_span: 14,
    timestamp: '2024-01-20',
    expenses: 48000
  }
  {
    _id: ObjectId('69b13a242c916e330847e3df'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [
      'grass',
      'leaves'
    ],
    care_taker_name: 'Priya',
    life_span: 60,
    timestamp: '2023-12-15',
    expenses: 75000
  }
```

```
{
  _id: ObjectId('69b13a242c916e330847e3e0'),
  animal_name: 'Bear',
  nature: 'harm',
  favorite_foods: [
    'honey',
    'fish'
  ],
  care_taker_name: 'Ravi',
  life_span: 25,
  timestamp: '2024-03-01',
  expenses: 35000
}
{
  _id: ObjectId('69b13a242c916e330847e3e1'),
  animal_name: 'Wolf',
  nature: 'harm',
  favorite_foods: [
    'rabbits',
    'meat'
  ],
  care_taker_name: 'Anil',
  life_span: 13,
  timestamp: '2024-02-15',
  expenses: 28000
}
> db.domestic_animals.find().pretty()
< {
  _id: ObjectId('69b13a872c916e330847e3e2'),
  animal_name: 'Dog',
  gender: 'male',
  favorite_foods: [
    'meat',
    'bones'
  ],
  animal_petname: 'Bruno',
  life_span: 12,
  timestamp: '2024-01-05',
  expenses: 15000
}
```

```
{
  _id: ObjectId('69b13a872c916e330847e3e3'),
  animal_name: 'Cat',
  gender: 'female',
  favorite_foods: [
    'fish',
    'milk'
  ],
  animal_petname: 'Whiskers',
  life_span: 15,
  timestamp: '2024-01-10',
  expenses: 12000
}
{
  _id: ObjectId('69b13a872c916e330847e3e4'),
  animal_name: 'Cow',
  gender: 'female',
  favorite_foods: [
    'grass',
    'hay'
  ],
  animal_petname: 'Lakshmi',
  life_span: 20,
  timestamp: '2023-11-20',
  expenses: 25000
}
{
  _id: ObjectId('69b13a872c916e330847e3e5'),
  animal_name: 'Rabbit',
  gender: 'male',
  favorite_foods: [
    'carrots',
    'grass'
  ],
  animal_petname: 'Bunny',
  life_span: 8,
  timestamp: '2024-02-01',
  expenses: 8000
}
```

```
{
  _id: ObjectId('69b13a872c916e330847e3e6'),
  animal_name: 'Horse',
  gender: 'male',
  favorite_foods: [
    'grass',
    'oats'
  ],
  animal_petname: 'Thunder',
  life_span: 25,
  timestamp: '2024-03-10',
  expenses: 40000
}
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
> db.domestic_animals.find(
      {},
      { animal_name: 1, expenses: 1, _id: 0 }
  ).pretty()
< {
    animal_name: 'Dog',
    expenses: 15000
  }
  {
    animal_name: 'Cat',
    expenses: 12000
  }
  {
    animal_name: 'Cow',
    expenses: 25000
  }
  {
    animal_name: 'Rabbit',
    expenses: 8000
  }
  {
    animal_name: 'Horse',
    expenses: 40000
  }
```

```
> db.wild_animals.find(
      {},
      { animal_name: 1, expenses: 1, _id: 0 }
  ).pretty()

< {
    animal_name: 'Tiger',
    expenses: 50000
  }
  {
    animal_name: 'Lion',
    expenses: 48000
  }
  {
    animal_name: 'Elephant',
    expenses: 75000
  }
  {
    animal_name: 'Bear',
    expenses: 35000
  }
  {
    animal_name: 'Wolf',
    expenses: 28000
  }
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
> db.domestic_animals.find({life_span: 12}).pretty()
< {
    _id: ObjectId('69b13a872c916e330847e3e2'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [
      'meat',
      'bones'
    ],
    animal_petname: 'Bruno',
    life_span: 12,
    timestamp: '2024-01-05',
    expenses: 15000
  }
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
> db.wild_animals.find({care_taker_name: "Ravi"}).pretty()
< {
    _id: ObjectId('69b13a242c916e330847e3dd'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [
      'deer',
      'meat'
    ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: '2024-02-10',
    expenses: 50000
  }
  {
    _id: ObjectId('69b13a242c916e330847e3e0'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [
      'honey',
      'fish'
    ],
    care_taker_name: 'Ravi',
    life_span: 25,
    timestamp: '2024-03-01',
    expenses: 35000
  }
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
db.domestic_animals.find(
  {life_span: {$gt: 5}},
  {animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0}
).pretty()
< {
  animal_name: 'Dog',
  favorite_foods: [
    'meat',
    'bones'
  ],
  expenses: 15000
}
{
  animal_name: 'Cat',
  favorite_foods: [
    'fish',
    'milk'
  ],
  expenses: 12000
}
{
  animal_name: 'Cow',
  favorite_foods: [
    'grass',
    'hay'
  ],
  expenses: 25000
}
{
  animal_name: 'Rabbit',
  favorite_foods: [
    'carrots',
    'grass'
  ],
  expenses: 8000
}
{
  animal_name: 'Horse',
  favorite_foods: [
    'grass',
    'oats'
  ],
  expenses: 40000
}
```

```
> db.wild_animals.find(
    {life_span: {$gt: 5}},
    {animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0}
  ).pretty()
< {
    animal_name: 'Tiger',
    favorite_foods: [
      'deer',
      'meat'
    ],
    expenses: 50000
  }
  {
    animal_name: 'Lion',
    favorite_foods: [
      'meat',
      'zebra'
    ],
    expenses: 48000
  }
  {
    animal_name: 'Elephant',
    favorite_foods: [
      'grass',
      'leaves'
    ],
    expenses: 75000
  }
  {
    animal_name: 'Bear',
    favorite_foods: [
      'honey',
      'fish'
    ],
    expenses: 35000
  }
  {
    animal_name: 'Wolf',
    favorite_foods: [
      'rabbits',
      'meat'
    ],
    expenses: 28000
  }
```